

To be defined

Steylor Barros¹[?????] and Andrey Brito²[0000-0002-7350-8599]

Federal University of Campina Grande, Campina Grande, PB 58429-900, BRA

`steylor.barros@ccc.ufcg.edu.br`

`andrey@computacao.ufcg.edu.br`

<https://www.computacao.ufcg.edu.br/>

Abstract. Keywords: Device Attestation, User Authentication, Zero Trust Security, Virtual Private Network (VPN), Remote Attestation, SPIFFE, SPIRE, Multi-Factor Authentication (MFA)

1 Introduction

Confidential data marketplaces must simultaneously support asset exchange, access control, and third-party computation over sensitive inputs. That combination creates a difficult systems problem. A marketplace that accepts externally supplied applications cannot treat code execution as an isolated implementation detail, because the same platform also handles persistent metadata, payment and access logic, storage backends, and the datasets that motivate the marketplace in the first place. In practice, the problem is not merely how to “run code safely,” but how to prevent build-time and run-time compromise from expanding into a full platform compromise.

The most direct implementation path would be a centralized confidential virtual machine [1] that hosts the marketplace-facing logic, local artifact storage, and the execution orchestrator in the same trust domain. That design is operationally attractive because it minimizes deployment complexity and avoids inter-service coordination. However, it also creates the wrong security shape for the problem: the same node that validates access and manages persistent sensitive state also becomes the place where untrusted or semi-trusted workloads are prepared and executed.

This paper argues that such a concentration of privilege is fundamentally ill-suited for high-assurance marketplaces. It is proposed an architecture designed under the assumption that component compromise is an eventual certainty rather than a remote possibility. By decomposing the marketplace into distinct trust domains—control, build, and execution [4]—the system ensures that a failure or malicious exploit in one phase cannot propagate to the entire application. Our focus is on minimizing the “blast radius” of errors and attacks, ensuring that even if the build or execution planes are corrupted, the platform’s core sensitive assets and persistent state remain protected.

2 Background

2.1 Confidential computing

Confidential Computing [1] is a paradigm that focuses on protecting data while it is being processed. It leverages hardware-level memory encryption to make a process or an entire Virtual Machine inaccessible to the host hypervisor. By using an ASID (Address Space Identifier), the processor applies cryptography to a memory context; the ciphertext is decrypted transparently by a key stored directly in the memory controller when the VM that owns that context accesses the data. Confidential Virtual Machines are recognized by a special bit, called the C-bit, which instructs the CPU to restrict access to the corresponding memory pages. Any attempt by the hypervisor to map those pages into its own address space raises a processor exception, enforcing hardware-backed isolation even against a privileged adversary.

2.2 Blockchain

A blockchain [5] is a peer-to-peer distributed system that records transactions with strong integrity guarantees. Transactions are grouped into blocks; each block contains a cryptographic hash of the previous block, creating a chain in which every entry implicitly attests to the entire preceding history. Altering any block invalidates all subsequent hashes, so corrupting the ledger requires recomputing more than fifty percent of the combined processing capacity of all participating peers—an effort generally considered computationally infeasible in practice. Blockchains also support Smart Contracts [3], which are programs deployed and executed directly on the network. A Smart Contract can register and govern access to relevant data, such as the encryption keys and access permissions used in our system. The main drawback of blockchains is their inherent transaction cost: every write operation requires consensus among the majority of peers before a block is accepted, which introduces both financial cost and latency.

2.3 IPFS

The InterPlanetary File System (IPFS) [2] addresses the challenge of storing large data volumes that do not integrate well with a blockchain due to the costs described above. IPFS is a structured distributed system that uses a Distributed Hash Table (DHT) to locate files across the network. Files are identified by their content hash—a deterministic, one-way function that produces a unique identifier for a given sequence of bytes; any modification to the content produces a different hash. The DHT ensures that data remains accessible to all peers by managing replication and routing efficiently across the network, making IPFS a practical complement to blockchain-based access control for large binary artifacts.

3 Problem definition and solution requirements

The core challenge addressed by the Dataset and Application Marketplace (DNAT) involves a transaction between two mutually distrusting parties: a *data buyer*, who owns a confidential application, and a *dataset provider*, who holds sensitive data. The objective is to enable a computation in which the buyer’s application processes the provider’s dataset without exposing either asset to the other party or to the underlying cloud infrastructure.

A straightforward implementation would deploy a single Confidential Virtual Machine (CVM) responsible for all platform functions: receiving uploads, managing the IPFS storage layer, interacting with the Smart Contract, resolving and installing Python dependencies, and running the application against the dataset. While operationally simple, this monolithic design concentrates all privileges within one trust domain. The critical vulnerability emerges during dependency resolution: installing external packages to satisfy an application’s requirements necessarily executes arbitrary third-party code with the same privileges as the marketplace layer, the IPFS node, and the blockchain client. A maliciously crafted package—or a compromised dependency in the supply chain—could exploit this phase to exfiltrate stored datasets, tamper with marketplace metadata, or establish a persistent backdoor affecting future executions. A build-time compromise thus propagates immediately into a full platform compromise, making dependency resolution a disproportionately dangerous entry point.

This observation reframes the core problem. Secure execution of the application at runtime is necessary but not sufficient. An equally important concern is preventing a compromise during the build and dependency resolution phase from reaching the broader platform. The problem must therefore be treated as a *privilege separation problem*: distinct operational phases—control, build, and execution—carry fundamentally different risk profiles and must occupy separate trust domains with different network-access policies, persistence rights, and failure blast radii.

Three design consequences follow from this reframing. First, the component managing marketplace state, blockchain interaction, and IPFS operations must never directly execute dependency installation logic for user-supplied applications. Second, the build phase, where external network access is unavoidable, must be isolated from both persistent sensitive storage and the execution runtime. Third, the execution phase, during which the sensitive dataset is consumed, must operate without any outbound network interface, eliminating the primary exfiltration channel even under a complete runtime compromise.

A secure solution must satisfy the following requirements:

4 Proposal

4.1 Architecture

The DNAT platform is organized around three Confidential Virtual Machines, each assigned a distinct operational role and a distinct set of persistence rights.

CVM1 acts as the control plane, **CVM2** as the isolated build plane, and **CVM3** as the isolated execution plane. The central design principle is privilege separation: a compromise in one plane must not grant access to the assets of the others. (...)

4.2 Implementation

The architecture described above is realized using Firecracker microVMs [?] as the primary isolation primitive for both the build and execution environments. The current prototype instantiates each CVM as a containerized service for development purposes; in a production deployment, each service would run inside a hardware-backed Confidential VM [1]. (...)

5 Evaluation

5.1 Security evaluation

5.2 Resource Consumption and Time Overhead

6 Related Work

7 Concluding Remarks

References

1. Advanced Micro Devices, Inc.: AMD SEV-SNP: Strengthening VM isolation with integrity protection and more. Tech. rep., Advanced Micro Devices, Inc. (2020)
2. Benet, J.: IPFS - content addressed, versioned, P2P file system. CoRR **abs/1407.3561** (2014), <https://arxiv.org/abs/1407.3561>
3. Buterin, V.: A next-generation smart contract and decentralized application platform. Ethereum White Paper (2014), <https://ethereum.org/en/whitepublication/>
4. Jr, J.R.N., Nunes, J.B.S., Falcão, E.L., Sampaio, L., Brito, A.: On the tracking of sensitive data and confidential executions. In: Proceedings of the 14th ACM International Conference on Distributed and Event-based Systems (DEBS '20). pp. 1–10. ACM, Virtual Event, QC, Canada (2020). <https://doi.org/10.1145/3401025.3404097>
5. Nakamoto, S.: Bitcoin: A peer-to-peer electronic cash system. White Paper (2008), <https://bitcoin.org/bitcoin.pdf>